

TODOTEC v3

Documentación técnica y manual de mantenimiento

ERP + e-commerce para Glowi Skin

Versión del documento	1.0
Código documentado	rama main · commit ac66047
Fecha de revisión	11 de junio de 2026
Audiencia	Programadores, soporte técnico y responsables de despliegue

Fuente de verdad: Este documento fue elaborado leyendo el código vigente. Los PDF históricos de 00_backup no se usaron como especificación funcional.

Cómo usar este documento

Este manual explica cómo está construido TodoTec v3 y dónde debe intervenir un programador para modificarlo sin romper inventario, pagos, contabilidad o despliegues. Las rutas de archivo son relativas a la raíz del repositorio `todotec_v3`.

Alcance: Describe el estado del código en el commit `ac66047`. Si el código cambia, actualizar este documento junto con la modificación.

Lectura recomendada

Perfil	Secciones prioritarias
Nuevo desarrollador	1, 2, 3, 7 y 12
Backend	4, 7, 8, 9, 10 y 13
Frontend admin	5, 8, 11 y 12
Frontend tienda	6, 8, 11 y 12
DevOps / soporte	3, 9, 10, 13 y 14

Contenido

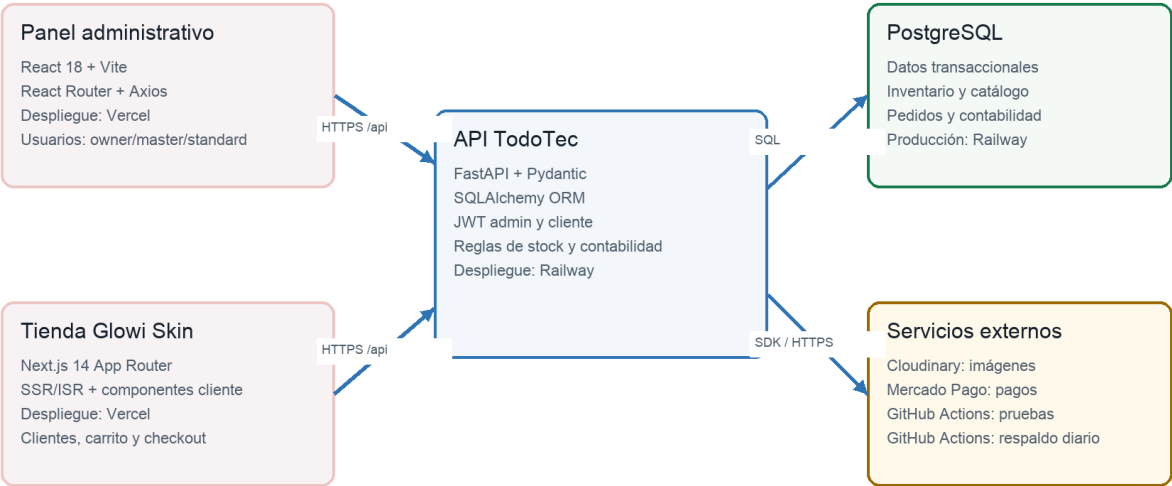
- 1. Resumen ejecutivo y arquitectura
- 2. Estructura del repositorio
- 3. Tecnologías y topología de despliegue
- 4. Backend FastAPI
- 5. Panel administrativo React
- 6. Tienda Next.js
- 7. Modelo de datos
- 8. Flujos críticos de negocio
- 9. Seguridad, autenticación y permisos
- 10. Configuración e integraciones
- 11. Guía de modificación por funcionalidad
- 12. Desarrollo local y pruebas
- 13. Despliegue, respaldo y recuperación
- 14. Diagnóstico, rendimiento y deuda técnica
- 15. Apéndices: endpoints, archivos y checklist

1. Resumen ejecutivo y arquitectura

TodoTec v3 es un sistema integrado de administración y venta para Glowi Skin. Combina un ERP operativo, un catálogo e-commerce y una API central. El backend es la única capa autorizada para leer o escribir la base de datos y concentra las reglas de negocio.

Arquitectura de ejecución de TodoTec v3

Tres aplicaciones desplegables, una base de datos y servicios externos



El admin y la tienda no acceden directamente a la base de datos.

Responsabilidades por aplicación

Componente	Responsabilidad	Entrada principal
backend/	API, autenticación, reglas de stock, ventas, compras, pedidos, contabilidad e integraciones.	backend/app/main.py
admin/	Panel interno para operar la empresa. Consume /api con JWT administrativo.	admin/src/main.jsx y admin/src/App.jsx
store/	Tienda pública, catálogo, cuenta del cliente, carrito y checkout.	store/app/layout.js y store/app/page.js

Principios que un mantenedor debe respetar

- Toda modificación de stock debe ejecutarse en el backend y dentro de la misma transacción que su operación comercial.
- Toda operación monetaria relevante debe producir asientos contables o una reversión auditable.
- Los usuarios administrativos y los clientes de la tienda usan autenticaciones separadas.
- Las imágenes se guardan en Cloudinary; la base de datos conserva sus URL.
- Los secretos pertenecen a variables de entorno, nunca al repositorio.

2. Estructura del repositorio

```

todotec_v3/
├── backend/                                API FastAPI y pruebas
│   ├── app/
│   │   ├── main.py                       arranque, CORS, routers y migraciones
│   │   ├── database.py                   conexión SQLAlchemy
│   │   ├── models.py                    tablas y relaciones
│   │   ├── schemas.py                   contratos Pydantic
│   │   ├── utils.py                     reversión contable
│   │   └── routers/                     endpoints por dominio
│   ├── tests/                           pruebas vitales
│   ├── requirements.txt
│   └── railway.json
├── admin/                                panel React + Vite
│   ├── src/api/                          cliente HTTP
│   ├── src/context/                      sesión y tema
│   ├── src/components/                  layout y componentes globales
│   └── src/pages/                       módulos operativos
├── store/                                tienda Next.js
│   ├── app/                             rutas App Router
│   ├── components/                     navegación, banners, productos y carrito
│   └── lib/                             API, sesión y persistencia de carrito
├── .github/workflows/                   pruebas y respaldo automático
├── 00_backup/                          material histórico, no ejecutable
└── docs/                              documentación técnica vigente

```

Archivos que no deben editarse ni versionarse

Ruta/patrón	Motivo
admin/node_modules/, store/node_modules/	Dependencias instaladas; se reconstruyen con npm install.
admin/dist/, store/.next/	Artefactos de compilación generados.
backend/.venv/	Entorno virtual local.
backend/todotec.db	Base SQLite local; las pruebas la eliminan y recrean.
__pycache__/, *.pyc, .DS_Store, .Rhistory	Archivos temporales del sistema o de herramientas.
00_backup/	Referencia histórica. No participa en ejecución ni despliegue.

Importante: El .gitignore ya excluye estos artefactos. Si aparecen en Git, revisar antes de borrarlos para no confundir datos locales con código.

Mapa rápido: quiero cambiar...

Objetivo	Comenzar en
Texto o bloques de la portada	store/app/page.js y admin/src/pages/Apariencia.jsx
Campos de producto	backend/app/models.py, schemas.py, routers/products.py, admin/src/pages/Inventario.jsx, store/app/product/[id]/page.js

Objetivo	Comenzar en
Proceso de compra	backend/app/routers/purchases.py y admin/src/pages/Compras.jsx
Proceso de venta	backend/app/routers/sales.py y admin/src/pages/Ventas.jsx
Checkout/pagos	store/app/checkout/page.js y backend/app/routers/orders.py
Usuarios/permisos	backend/app/routers/auth.py, admin/src/context/AuthContext.jsx y admin/src/pages/Usuarios.jsx
Navegación del admin	admin/src/App.jsx y admin/src/components/Layout.jsx

3. Tecnologías y topología de despliegue

Capa	Tecnología	Versión observada	Uso
API	Python / FastAPI	FastAPI 0.111.0	Endpoints HTTP, validación y OpenAPI
Servidor	Uvicorn	0.29.0	Proceso ASGI en Railway
ORM	SQLAlchemy	2.0.30	Persistencia y relaciones
Validación	Pydantic	2.7.1	Contratos de entrada/salida
BD	PostgreSQL / SQLite	Producción / fallback local	Datos operativos
Admin	React + Vite	React 18.3 / Vite 5.2	SPA interna
Tienda	Next.js App Router	14.2.3	SSR/ISR y frontend público
CSS	Tailwind CSS	3.4.4	Estilos admin y tienda
HTTP	Axios / fetch	Axios 1.7.2	Consumo de API
Imágenes	Cloudinary	SDK 1.40.0	Carga y CDN
Pagos	Mercado Pago	SDK 2.2.2	Checkout Pro y QR
CI/backup	GitHub Actions	Workflows YAML	Pruebas y pg_dump diario

Despliegue actual configurado en código

Servicio	Plataforma	Configuración
Backend	Railway	backend/railway.json inicia uvicorn app.main:app --host 0.0.0.0 --port \$PORT
Admin	Vercel	admin/vercel.json redirige /api y /uploads al backend Railway
Tienda	Vercel	store/next.config.js usa NEXT_PUBLIC_API_URL y rewrites
Base de datos	Railway PostgreSQL	DATABASE_URL inyectada como variable de entorno

URLs: El código contiene <https://todotec-v3-production.up.railway.app> y <https://glowi-skin.vercel.app>. Confirmarlas en Railway/Vercel antes de una migración de dominio.

Ruta de una petición

1. El navegador llama /api/... desde el admin o la tienda.
2. Vercel reescribe la solicitud hacia el backend cuando corresponde.

3. FastAPI valida autenticación y datos Pydantic.
4. El router ejecuta reglas de negocio mediante una sesión SQLAlchemy.
5. SQLAlchemy confirma o revierte la transacción en PostgreSQL.
6. La respuesta JSON vuelve al frontend, que actualiza la interfaz.

4. Backend FastAPI

Arranque y ciclo de vida

Archivo	Responsabilidad
backend/app/main.py	Configura logging, crea tablas, ejecuta migraciones, CORS, routers, health check y reset total.
backend/app/database.py	Resuelve DATABASE_URL, crea engine, SessionLocal y dependencia get_db.
backend/app/models.py	Define tablas, relaciones y propiedades calculadas.
backend/app/schemas.py	Define entradas y salidas tipadas para operaciones principales.
backend/app/utlis.py	Genera asientos contables inversos para anulaciones.
backend/app/cloudinary_client.py	Configura el SDK de Cloudinary.

Base de datos y migraciones

Al importar backend/app/main.py se ejecuta `Base.metadata.create_all(bind=engine)` y luego `_run_migrations()`. Las migraciones son sentencias SQL almacenadas en una lista y cada una corre en su propia transacción; si falla, el sistema registra una advertencia y continúa.

Riesgo de mantenimiento: Las migraciones incluyen sintaxis PostgreSQL (SERIAL, ALTER COLUMN TYPE, ON CONFLICT). SQLite sirve como fallback y para pruebas, pero no representa completamente producción.

- No modificar una migración ya ejecutada esperando que se repita con efecto distinto.
- Para un nuevo campo: agregarlo al modelo y añadir una migración idempotente con IF NOT EXISTS.
- Probar primero contra una copia o entorno de staging PostgreSQL.
- Para cambios complejos conviene adoptar Alembic; hoy no está instalado.

Organización de routers

Dominio	Prefijo	Acceso principal	Archivo
Autenticación admin	/api/auth	Mixto; login público, gestión con roles	routers/auth.py
Proveedores	/api/suppliers	Admin	routers/suppliers.py
Productos	/api/products	GET público; mutaciones admin	routers/products.py
Compras	/api/purchases	Admin	routers/purchases.py
Ventas	/api/sales	Admin	routers/sales.py
Contabilidad	/api/accounting	Admin	routers/accounting.py
Cobranzas	/api/cobranzas	Admin	routers/cobranzas.py

Dominio	Prefijo	Acceso principal	Archivo
Dashboard	/api/dashboard	Admin	routers/dashboard.py
Categorías/marcas/necesidades	/api/categories brands necessities	GET público; mutaciones admin	routers correspondientes
Apariencia	/api/appearance	Lectura pública; edición admin	routers/appearance.py
Pedidos	/api/orders	Checkout público; gestión admin	routers/orders.py
Cientes tienda	/api/customers	JWT de cliente	routers/customers.py
Cientes admin	/api/clients	Admin	routers/clients.py
Revalidación	/api/revalidate-store	Admin	routers/revalidate.py

Patrón de implementación de un endpoint

```
@router.post("", response_model=EntidadOut)
def crear(data: EntidadCreate, db: Session = Depends(get_db)):
    # 1. validar reglas de negocio
    # 2. crear/modificar modelos
    # 3. db.flush() si se necesita el ID
    # 4. crear stock/asientos relacionados
    # 5. db.commit() y db.refresh(entidad)
    return entidad
```

Transacciones: Ante una excepción posterior a una escritura, ejecutar `db.rollback()` o permitir que el bloque de manejo del router revierta antes de devolver el error.

Contratos Pydantic

`schemas.py` contiene los contratos compartidos de configuración, proveedores, productos, compras, ventas, cobranzas y contabilidad. Algunos routers, como `auth.py`, `customers.py` y `orders.py`, definen esquemas locales. Si se añade un campo, verificar tanto el modelo SQLAlchemy como todos los esquemas Create, Update y Out involucrados.

5. Panel administrativo React

El admin es una SPA de React construida con Vite. React Router controla las pantallas y Axios centraliza las llamadas. El token se guarda en localStorage bajo admin_token.

Elemento	Archivo	Función
Bootstrap	admin/src/main.jsx	Monta React, Router y estilos.
Rutas	admin/src/App.jsx	Login, ruta protegida y páginas internas.
Layout	admin/src/components/Layout.jsx	Sidebar, navegación, usuario, logout y revalidación.
Cliente API	admin/src/api/client.js	Axios, helpers, compresión de imágenes e interceptor 401.
Sesión	admin/src/context/AuthContext.jsx	Login, restauración de token, usuario y logout.
Tema	admin/src/context/ThemeContext.jsx	Lee colores, empresa y logo desde /api/config.
Carga global	admin/src/components/GlobalLoadingOverlay.jsx	Bloquea dobles mutaciones mientras hay POST/PUT/PATCH/DELETE.

Módulos visibles

Ruta	Página	Backend principal
/	Dashboard.jsx	/dashboard
/proveedores	Proveedores.jsx	/suppliers
/inventario	Inventario.jsx	/products
/compras	Compras.jsx	/purchases
/ventas	Ventas.jsx	/sales
/cobranzas	Cobranzas.jsx	/cobranzas
/contabilidad	Contabilidad.jsx	/accounting
/etiquetas	Etiquetas.jsx	/categories, /brands, /necessities
/apariencia	Apariencia.jsx	/appearance
/pedidos	Pedidos.jsx	/orders
/clientes	Clientes.jsx	/clients
/configuracion	Configuracion.jsx	/config y /reset-total
/usuarios	Usuarios.jsx	/auth/users; solo master/owner

Patrón para agregar un módulo

1. Crear el router/backend y registrarlo en backend/app/main.py.
2. Agregar helpers HTTP en admin/src/api/client.js o usar el cliente api central.
3. Crear admin/src/pages/NuevoModulo.jsx.
4. Registrar la ruta en admin/src/App.jsx.
5. Agregar el acceso a NAV_BASE o NAV_MASTER en admin/src/components/Layout.jsx.
6. Aplicar permisos en backend; ocultar el menú no reemplaza la seguridad del servidor.

Detalle importante del cliente HTTP

- baseUrl es /api; Vite usa proxy en desarrollo y Vercel usa rewrites en producción.
- El interceptor agrega seguimiento de mutaciones para evitar dobles clics.
- Un 401 elimina admin_token y redirige a /login.
- Las imágenes grandes se comprimen en navegador a JPEG antes de subirlas.

6. Tienda Next.js

La tienda usa Next.js 14 con App Router. Las páginas públicas consumen el backend mediante `store/lib/api.js`. Los módulos de sesión y carrito son componentes cliente y usan `localStorage`.

Ruta	Archivo	Responsabilidad
/	<code>store/app/page.js</code>	Banners, promociones y secciones de productos.
/catalog	<code>store/app/catalog/page.js</code>	Catálogo y filtros.
/product/[id]	<code>store/app/product/[id]/page.js</code>	Ficha, variantes, contenido y carrito.
/cart	<code>store/app/cart/page.js</code>	Resumen del carrito y cupón.
/checkout	<code>store/app/checkout/page.js</code>	Cliente, entrega, pago MP/QR y creación de pedido.
/checkout/success	<code>store/app/checkout/success/page.js</code>	Confirmación/consulta posterior al pago.
/login y /register	<code>store/app/login register/page.js</code>	Cuenta independiente del admin.
/account	<code>store/app/account/page.js</code>	Datos, contraseña, cupones y carrito persistente.
/contact	<code>store/app/contact/page.js</code>	Contacto y puntos de entrega.
/privacidad	<code>store/app/privacidad/page.js</code>	Política de privacidad.
/api/revalidate	<code>store/app/api/revalidate/route.js</code>	Revalidación protegida por secreto.

Datos y caché

- `store/lib/api.js` usa `fetch` con `cache: force-cache` para catálogo, configuración y apariencia.
- El botón Actualizar Tienda del admin llama `/api/revalidate-store`.
- El backend llama la ruta interna `/api/revalidate` de la tienda usando `REVALIDATE_SECRET`.
- Sin revalidación, un cambio de catálogo puede tardar en verse por la caché.

Sesión y carrito

Dato local	Clave	Módulo
Token de cliente	<code>glowi_token</code>	<code>store/lib/customer.js</code>
Datos de cliente	<code>glowi_customer</code>	<code>store/lib/customer.js</code>
Cupón aplicado	<code>glowi_coupon</code>	<code>store/lib/customer.js</code>
Carrito local	<code>cart</code>	<code>store/lib/cartStorage.js</code>

Cuando un cliente inicia sesión, `mergeServerCart` une el carrito local con el almacenado en la tabla `customers`. Las cuentas del cliente usan un JWT distinto al del panel administrativo.

Componentes globales

Componente	Función
Navbar.js	Navegación, cuenta y acceso al carrito.
WelcomeBanner.js	Franja de anuncio.
CartDrawer.js	Carrito lateral global.
Prefetcher.js	Precarga de navegación/datos.
BannerCarousel.js	Carrusel administrable.
ProductCard.js	Representación reutilizable de producto.
Footer.js	Contacto, redes y enlaces.

7. Modelo de datos

El modelo se concentra en backend/app/models.py. La siguiente agrupación sirve para entender propiedad y dependencia de la información.

Grupo	Tablas	Relaciones clave
Configuración	company_config	Registro principal con marca, impuestos, correlativo, redes y QR.
Catálogo	products, product_variants, product_images, categories, brands, necessities	Producto tiene variantes e imágenes; necesidad usa FK; categoría y marca se conservan como texto.
Abastecimiento	suppliers, purchases, purchase_items	Compra pertenece a proveedor y contiene ítems; incrementa variantes.
Ventas	sales, sale_items	Venta contiene ítems y puede originarse manualmente o desde Order.
Contabilidad	accounting_entries, capital_contributions, payments	Asientos enlazan compra, venta o capital; pagos liquidan créditos.
E-commerce	orders, customers, coupons	Pedido guarda snapshot JSON de ítems y entrega; puede generar una Sale.
Apariencia	banners, homepage_sections	Contenido administrable de portada.
Seguridad	users	Usuarios del admin con roles.

Entidades esenciales

Entidad	Campos que gobiernan el comportamiento
Product	sku, category, brand, necessity_id, sale_price, ref_cost, is_active, show_in_store, view_count
ProductVariant	product_id, color, stock, image_url
Purchase	supplier_id, costos, crédito, vencimiento, status
Sale	cliente, montos, crédito, invoice_number, sale_type, status
AccountingEntry	entry_type, debit_account, credit_account, amount y referencias
Order	status, cliente, delivery_type/data, items JSON, montos, pago, sale_id, cupón
User	username, password_hash, role, is_active
Customer	email, perfil, delivery_data, cart, is_active

Decisiones de modelado que afectan cambios futuros

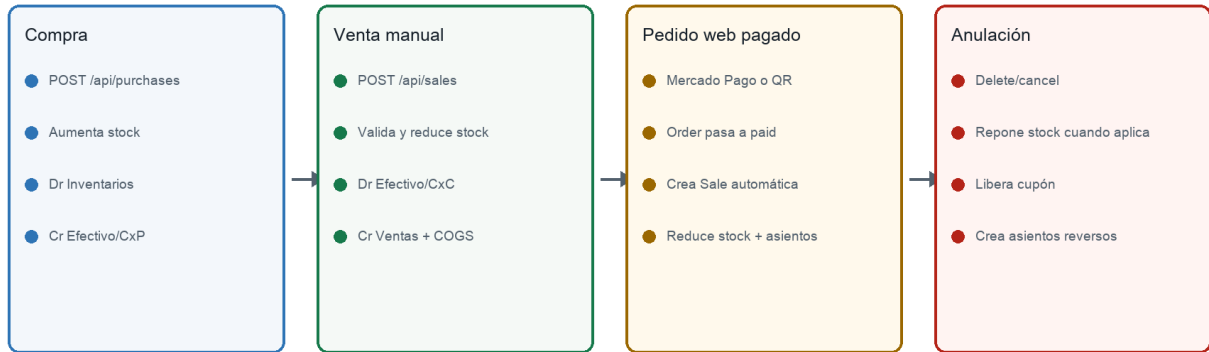
- category y brand están almacenados como texto en products; al renombrar, los routers propagan el valor.
- necessity_id sí es una clave foránea hacia necessities.

- Order.items es un snapshot JSON para preservar lo comprado aunque el producto cambie después.
- Los importes persistentes usan Numeric, pero varios esquemas Pydantic y cálculos convierten a float.
- Product.unit_cost es un costo promedio calculado desde purchase_items, no una columna almacenada.
- Product.total_stock suma las variantes en Python.

Precisión monetaria: Para nuevas reglas financieras, preferir Decimal de extremo a extremo. La mezcla actual Numeric/float puede introducir redondeos en casos límite.

8. Flujos críticos de negocio

Flujos que modifican inventario y contabilidad



Regla de mantenimiento: una nueva operación que cambie dinero o stock debe definir también su reversión y sus pruebas.

Compra e ingreso de inventario

1. El admin envía PurchaseCreate a POST /api/purchases.
2. El backend crea o localiza productos y variantes.
3. Distribuye costos, registra Purchase/PurchaseItem e incrementa stock.
4. Si es contado, acredita Efectivo; si es crédito, acredita Cuentas por Pagar.
5. Una rectificación recalcula diferencias de cantidad/costo y ajusta inventario/asientos.

Archivos: backend/app/routers/purchases.py, backend/app/schemas.py, admin/src/pages/Compras.jsx.

Venta manual

1. POST /api/sales valida existencia y stock.
2. Crea Sale y SaleItem, asigna correlativo y descuenta variantes.
3. Registra ingreso contra Efectivo o Cuentas por Cobrar.
4. Registra costo de ventas contra Inventarios.
5. Al borrar, debe reponer stock y crear asientos reversos.

Archivos: backend/app/routers/sales.py, backend/app/utils.py, admin/src/pages/Ventas.jsx.

Pedido web y pago

1. El checkout envía ítems, cliente, entrega y cupón.
2. _validate_stock rechaza faltantes antes de cobrar.
3. El servidor valida el cupón y recalcula descuento, IGV y total.
4. Mercado Pago crea una preferencia o se crea un pedido QR pendiente.
5. Webhook/verificación o el admin cambia el pedido a paid.

6. `_create_sale_from_order` crea la venta, descuenta stock y genera asientos.

7. El pedido conserva `sale_id` para evitar duplicar la venta.

Archivos: `store/app/checkout/page.js` y `backend/app/routers/orders.py`.

Crédito y cobranzas

Compras y ventas pueden ser a crédito. La tabla `payments` registra abonos. Los endpoints de cobranzas calculan total pagado, saldo y estado; cada abono también debe reflejar el movimiento entre Efectivo y Cuentas por Pagar/Cobrar.

Cancelaciones y reversión

Regla crítica: No borrar asientos históricos como forma de anular una operación. `create_reversal_entries` invierte débito y crédito para conservar trazabilidad.

9. Seguridad, autenticación y permisos

Administración

Aspecto	Implementación
Algoritmo	JWT HS256; expiración de 12 horas.
Secreto	JWT_SECRET. Si falta, se genera uno temporal y las sesiones se invalidan al redeploy.
Contraseñas	bcrypt mediante passlib.
Token frontend	localStorage: admin_token.
Usuario inicial	Solo si users está vacía; INITIAL_ADMIN_USER e INITIAL_ADMIN_PASSWORD.
Normalización login	strip() + lower() para el nombre de usuario.

Jerarquía de roles

Rol	Capacidad
standard	Opera módulos normales protegidos.
master	Incluye standard; puede listar y crear usuarios, y administrar su contraseña.
owner	Incluye master; puede borrar usuarios, cambiar contraseñas ajenas y ejecutar reset total.

Autorización real: Los permisos se aplican con Depends(get_current_user), require_master y require_owner en el backend. El menú del frontend es solo presentación.

Clientes de la tienda

- Autenticación independiente en backend/app/routers/customers.py.
- El token incluye el tipo customer y no es válido para endpoints del admin.
- La sesión dura 30 días.
- El carrito y los cupones quedan asociados al customer_id.

Superficie pública

- Lectura de productos, categorías, marcas, necesidades, configuración, banners y secciones.
- Registro/login de clientes y operaciones autenticadas de su cuenta.
- Creación de preferencia o pedido QR, webhook y verificación de pago.
- Las mutaciones administrativas de catálogo y apariencia requieren JWT admin.

Controles recomendados al modificar

- Nunca aceptar total, descuento o estado de pago como verdad exclusiva del navegador.
- No registrar contraseñas, tokens, secretos ni payloads sensibles en logs.
- Validar ownership: un cliente solo puede usar sus cupones y datos.
- Mantener idempotencia en webhook/verificación para no generar ventas duplicadas.
- Usar bloqueos o validación transaccional al descontar stock.

10. Configuración e integraciones

Variables de entorno del backend

Variable	Obligatoria	Uso
DATABASE_URL	Producción	Conexión PostgreSQL; sin ella usa SQLite local.
JWT_SECRET	Sí	Firma estable de sesiones administrativas y de clientes.
INITIAL_ADMIN_USER	Instalación inicial	Nombre del primer owner si users está vacía.
INITIAL_ADMIN_PASSWORD	Instalación inicial	Contraseña del primer owner.
ADMIN_URL	Producción	Origen CORS del panel.
STORE_URL	Producción	Origen CORS, retornos de pago y revalidación.
BACKEND_URL	Pagos	URL pública para webhook de Mercado Pago.
RAILWAY_PUBLIC_DOMAIN	Alternativa	Fallback de BACKEND_URL.
MP_ACCESS_TOKEN	Mercado Pago	Credencial del SDK.
MP_SANDBOX	Según entorno	true para pruebas; false para cobros reales.
CLOUDINARY_CLOUD_NAME	Imágenes	Cuenta Cloudinary.
CLOUDINARY_API_KEY	Imágenes	Clave pública del SDK.
CLOUDINARY_API_SECRET	Imágenes	Secreto del SDK.
REVALIDATE_SECRET	Sí	Firma compartida para purgar caché de tienda.

Variables del frontend

Aplicación	Variable	Uso
Admin	VITE_API_URL	Proxy de desarrollo en admin/vite.config.js.
Tienda	NEXT_PUBLIC_API_URL	Backend usado por fetch y rewrites.
Tienda	REVALIDATE_SECRET	Protege store/app/api/revalidate/route.js.

Cloudinary

Productos, variantes, logo, banners, QR y comprobantes se cargan mediante el backend. Para cambiar carpetas o transformaciones, revisar cloudinary_client.py y los endpoints de products.py, config.py, appearance.py y orders.py.

Mercado Pago

orders.py construye preferencias, define URLs de retorno y webhook, verifica pagos y transforma un pedido pagado en venta. Antes de activar producción comprobar MP_SANDBOX=false, credenciales, BACKEND_URL HTTPS y recepción pública del webhook.

Revalidación de tienda

```
Admin: POST /api/revalidate-store
Backend: POST {STORE_URL}/api/revalidate
Header/body: secreto compartido REVALIDATE_SECRET
Tienda: revalidatePath('/') y rutas relacionadas
```

11. Guía de modificación por funcionalidad

A. Cambiar un texto fijo de la portada

1. Buscar el texto con rg: rg -n "texto actual" store admin backend.
2. Si está en store/app/page.js, editar el JSX correspondiente.
3. Ejecutar npm run build dentro de store/.
4. Publicar y comprobar móvil/escritorio.

Ejemplo actual: "Puntos para recojo gratis" está en store/app/page.js.

B. Cambiar banners o secciones sin tocar código

1. Usar Apariencia en el admin.
2. El admin actualiza /api/appearance.
3. Pulsar Actualizar Tienda para revalidar la caché.

Código relacionado: admin/src/pages/Apariencia.jsx, routers/appearance.py, BannerCarousel.js.

C. Agregar un campo de producto

1. Agregar la columna a Product en backend/app/models.py.
2. Añadir una migración idempotente en _run_migrations de backend/app/main.py.
3. Agregar el campo a ProductCreate, ProductUpdate y ProductOut en schemas.py.
4. Confirmar que products.py permite crear/actualizar el valor.
5. Añadir control en admin/src/pages/Inventario.jsx.
6. Mostrarlo donde corresponda en store/app/product/[id]/page.js.
7. Agregar prueba de creación, edición y respuesta pública.

No olvidar: Si el campo afecta precio, stock o contabilidad, el cambio debe revisarse también en purchases.py, sales.py y orders.py.

D. Crear una nueva etiqueta o taxonomía

1. Definir modelo y tabla; decidir si Product guardará FK o texto.
2. Crear esquemas y router CRUD con GET público y mutaciones protegidas.
3. Registrar router en main.py.
4. Agregar pestaña en admin/src/pages/Etiquetas.jsx.
5. Agregar filtro en store/lib/api.js y store/app/catalog/page.js.
6. Definir qué ocurre al renombrar o eliminar una etiqueta usada.

E. Modificar entrega o puntos de recojo

Área	Archivo
------	---------

Área	Archivo
Opciones y formulario del checkout	store/app/checkout/page.js
Texto informativo/contacto	store/app/contact/page.js y store/app/page.js
Contrato de entrega	DeliveryIn en backend/app/routers/orders.py
Persistencia	Order.delivery_type y Order.delivery_data en models.py
Dirección copiada a la venta	_create_sale_from_order en orders.py

F. Modificar cálculo de precio, IGV o descuento

- Configuración general: CompanyConfig.tax_rate y admin Configuración.
- Venta manual: routers/sales.py.
- Pedido web/cupón: routers/orders.py y routers/customers.py.
- Presentación del total: store/app/cart/page.js y checkout/page.js.
- Pruebas: test_vitales.py debe validar importes exactos.

Seguridad: El backend debe recalcular siempre. Nunca confiar en el total enviado por el navegador.

G. Cambiar reglas de permisos

1. Modificar ROLE_RANK o dependencias en routers/auth.py.
2. Aplicar la dependencia al endpoint, no solo a la página.
3. Ajustar navegación de Layout.jsx y acciones de Usuarios.jsx.
4. Agregar pruebas 401/403 para cada rol.

H. Agregar un nuevo módulo administrativo

1. Diseñar modelo y reglas de negocio.
2. Crear router, esquemas y pruebas backend.
3. Registrar include_router en main.py con el nivel de acceso correcto.
4. Crear API helpers y página React.
5. Registrar ruta en App.jsx y menú en Layout.jsx.
6. Verificar build, permisos y estados de error/carga.

12. Desarrollo local y pruebas

Backend

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
uvicorn app.main:app --reload --port 8000
```

Comprobaciones: <http://localhost:8000/health> y <http://localhost:8000/docs>.

Admin

```
cd admin
npm install
npm run dev
# Vite: http://localhost:5173
```

Tienda

```
cd store
npm install
NEXT_PUBLIC_API_URL=http://localhost:8000 npm run dev
# Next.js: http://localhost:3000
```

Pruebas automatizadas

```
cd backend
source .venv/bin/activate
pip install pytest
python -m pytest tests/ -v
```

Las pruebas cubren protección de endpoints, jerarquía de roles, compra→venta→contabilidad, pedido QR, stock, cupón y propagación de categorías.

Precaución: `test_vitales.py` elimina `backend/todotec.db` al importar. No ejecutar esas pruebas apuntando a una base local que contenga datos importantes.

Builds obligatorios antes de integrar

```
cd admin && npm run build
cd ../store && NEXT_PUBLIC_API_URL=https://URL-BACKEND npm run build
```

- El build de tienda necesita un backend accesible para páginas que consultan datos durante prerender.
- El admin actualmente puede advertir que el bundle principal es grande; no impide el build, pero es deuda de rendimiento.

Checklist de una modificación

Paso	Verificación
Código	Cambio localizado, sin archivos generados ni secretos.

Paso	Verificación
Datos	Migración idempotente y respaldo previo si cambia esquema.
Backend	Pruebas vitales y /docs coherente.
Admin	Login, permisos, caso exitoso y error.
Tienda	Desktop/móvil, carrito y checkout si aplica.
Build	npm run build en ambos frontends.
Git	git diff revisado, commit descriptivo y push a main.

13. Despliegue, respaldo y recuperación

Flujo de despliegue

1. Crear un respaldo si el cambio modifica datos, migraciones, pagos o inventario.
2. Ejecutar pruebas backend y builds frontend.
3. Revisar git diff y confirmar que no hay .env, bases locales ni artefactos.
4. Hacer commit y push a main.
5. Railway/Vercel despliegan según su integración con el repositorio.
6. Revisar logs de arranque y migraciones.
7. Probar /health, login admin, catálogo y una operación no destructiva.
8. Pulsar Actualizar Tienda cuando cambien datos cacheados.

Respaldo automático

.github/workflows/backup.yml ejecuta pg_dump diariamente a las 09:00 UTC, equivalente a las 04:00 en Perú, y conserva el artefacto 30 días. Requiere el secret de GitHub DATABASE_PUBLIC_URL.

Pruebas continuas

.github/workflows/tests.yml corre las pruebas vitales en cada push a main y en pull requests usando Python 3.12.

Recuperación recomendada

1. Detener temporalmente escrituras o poner el sistema en mantenimiento.
2. Descargar el .dump correcto desde GitHub Actions.
3. Crear una base vacía o respaldo de la base actual antes de reemplazarla.
4. Restaurar con pg_restore en el servicio PostgreSQL autorizado.
5. Revisar migraciones del código desplegado contra la fecha del respaldo.
6. Verificar conteos de productos, ventas, pedidos, asientos y usuarios.
7. Probar login y conciliación contable antes de reabrir.

No improvisar: Una restauración sobrescribe estado comercial. Debe definirse el punto de recuperación y conservarse una copia del estado previo.

Rollback de código

Revertir el commit problemático y desplegar nuevamente. Un rollback de código no deshace automáticamente una migración ni datos escritos; los cambios de esquema necesitan un plan explícito de compatibilidad o restauración.

14. Diagnóstico, rendimiento y deuda técnica

Matriz de diagnóstico

Síntoma	Revisar primero	Causa frecuente
Login falla para una cuenta	Railway logs y /api/auth/login	Contraseña, usuario inactivo, token/secret o datos distintos entre entornos.
Sesiones caen tras despliegue	JWT_SECRET	Secreto ausente; el backend genera uno aleatorio.
Cambio no aparece en tienda	Actualizar Tienda y REVALIDATE_SECRET	Caché force-cache sin revalidar.
Imágenes no cargan	Cloudinary variables y URL guardada	Credenciales, dominio o carga fallida.
Pago no crea venta	Webhook, MP_ACCESS_TOKEN, BACKEND_URL y Order.status	Webhook inaccesible o pago no verificado.
Stock incorrecto	Sale/Order y variante elegida	Doble confirmación, operación manual o reversión incompleta.
Admin lento	Network y consultas del endpoint	Listados completos, N+1 y bundle grande.
Migración advierte al iniciar	Railway logs	SQL ya aplicado, incompatibilidad o esquema parcial.

Oportunidades de rendimiento priorizadas

Prioridad	Mejora	Impacto
Alta	Paginar products, sales, purchases, orders y clients; filtrar en servidor.	Menos datos, memoria y tiempo de render.
Alta	Usar selectinload/joinedload en respuestas con relaciones.	Evita consultas N+1 de productos, variantes e ítems.
Alta	Reescribir dashboard con agregaciones SQL y menos bucles.	Reduce consultas repetidas y cálculos Python.
Media	Optimizar clients para agregar pedidos por consulta agrupada.	Evita una consulta por cliente.
Media	Dividir el bundle admin con lazy imports por ruta.	Carga inicial más rápida; hoy existe advertencia de chunk grande.
Media	Añadir índices/medición según slow queries reales.	Mejora búsquedas sin indexar de más.
Baja	Cachear catálogos estables en admin con invalidación.	Reduce GET repetidos.

Deuda técnica y riesgos conocidos

- Migraciones manuales en main.py sin historial/versionado formal; adoptar Alembic.
- SQLite local no reproduce todas las migraciones ni bloqueos de PostgreSQL.

- Listados principales no tienen paginación.
- Schemas anidados pueden disparar N+1 y respuestas pesadas.
- Cálculos financieros mezclan Numeric y float.
- Order.sale_id y customer_id no están declarados como ForeignKey en el modelo.
- La raíz / devuelve "TodoTec API v2.0" aunque FastAPI declara versión 3.0.0.
- Hay archivos Categorías.jsx y Marcas.jsx no registrados como rutas; Etiquetas.jsx concentra esas funciones.
- No hay pruebas frontend ni pruebas end-to-end de Mercado Pago real.
- No existe staging documentado; los cambios sensibles requieren especial cautela.

Orden sugerido de modernización

1. Agregar observabilidad de tiempos y consultas lentas.
2. Pagar y eliminar N+1 en los endpoints más usados.
3. Separar migraciones con Alembic y crear entorno staging PostgreSQL.
4. Unificar Decimal para dinero y ampliar pruebas financieras.
5. Incorporar pruebas E2E de login, compra, venta y checkout sandbox.

15. Apéndices

A. Inventario de endpoints

Dominio	Operaciones	Acceso
auth	POST /login; GET /me; GET/POST /users; DELETE /users/{id}; PUT password; POST change-password	Mixto/roles
suppliers	GET lista/detalle; POST; PUT; DELETE	Admin
products	GET lista/detalle/categories; POST view; CRUD; variantes; imágenes	GET público, mutación admin
purchases	GET lista/detalle; POST; PATCH rectify; DELETE	Admin
sales	GET lista/detalle; POST; DELETE	Admin
accounting	entries, cash, capital CRUD, reset, income-statement, balance-sheet	Admin
dashboard	GET /api/dashboard	Admin
cobranzas	payables/pay y receivables/collect	Admin
categories	GET activas/all; POST; PUT; DELETE	GET público, mutación admin
brands	GET activas/all; POST; PUT; DELETE	GET público, mutación admin
necessities	GET activas/all; POST; PUT; DELETE	GET público, mutación admin
appearance	announcement, qr-image, banners, sections	Lectura pública, edición admin
orders	create-preference, webhook, verify, lista, QR, screenshot, manual, status, delete	Mixto
customers	register, login, me, password, coupons, cart	Cliente
clients	lista, cupones y summary	Admin
revalidate	POST /api/revalidate-store	Admin
system	GET /; GET /health; POST /api/reset-total	Público / owner

La especificación interactiva completa se genera automáticamente en /docs cuando el backend está activo.

B. Archivos por responsabilidad

Responsabilidad	Archivos principales
Arranque/API	backend/app/main.py, database.py
Datos	backend/app/models.py, schemas.py
Inventario	routers/products.py, purchases.py; admin Inventario.jsx, Compras.jsx
Ventas/contabilidad	routers/sales.py, accounting.py, cobranzas.py, utils.py
Pedidos/pagos	routers/orders.py; store checkout/page.js

Responsabilidad	Archivos principales
Seguridad	routers/auth.py, customers.py; AuthContext.jsx, customer.js
Apariencia	routers/appearance.py; admin Apariencia.jsx; store page.js y BannerCarousel.js
Despliegue	backend/railway.json, admin/vercel.json, store/next.config.js
Calidad	backend/tests/test_vitales.py, .github/workflows/tests.yml
Respaldo	.github/workflows/backup.yml

C. Definition of Done

- La modificación funciona en el flujo feliz y en errores previsibles.
- La autorización está aplicada en backend.
- Stock, dinero y contabilidad permanecen consistentes.
- Las migraciones son idempotentes y existe respaldo cuando corresponde.
- Pruebas backend y builds frontend pasan.
- No se incluyeron secretos, bases locales ni artefactos generados.
- Se revisaron logs y despliegue.
- Esta documentación se actualizó si cambió arquitectura, configuración o procedimiento.

D. Glosario

Término	Significado
ERP	Sistema interno para inventario, compras, ventas, créditos y contabilidad.
Admin	Aplicación privada usada por trabajadores.
Store	Tienda pública Glowi Skin.
Router	Módulo FastAPI que agrupa endpoints de un dominio.
Schema	Contrato Pydantic de entrada o salida.
ISR/revalidación	Actualización controlada de contenido cacheado por Next.js.
COGS	Costo de ventas; salida contable de inventario al vender.
Reversión	Asiento inverso que anula el efecto sin borrar el historial.
Owner	Máximo rol administrativo.

Fin del documento